Reduceri polinomiale

15 octombrie 2024

Numim **instanțe-da (engl. yes-instance)** ale unei probleme de decizie P instanțele (datele de intrare) pentru P care apartin multimii definite de P. De exemplu, un triunghi este o instanța-da pentru problema 3-CLIQUE (care primeste ca input un graf si accepta in cazul in care graful respectiv contine 3 noduri cu muchii intre fiecare 2 dintre ele). Un graf care nu contine nici un triunghi este o **instanța-nu** pentru problema 3-CLIQUE.

Reducere polinomială

PA: o problema pe care o reduc

PB: o problema la care reduc problema PA

F = algoritm **determinist polinomial** de transformare a instantelor problemei PA in instanța ale problemei PB Dacă există F algoritm determinist cu complexitate polinomială care transformă **orice** instantă I a problemei PA într-**o** instanță F(i) a problemei PB a.î

I este o instanța-da pentru PA <=> F(I) este o instanța-da pentru PB,

atunci spunem că PA se reduce polinomial la PB: **PA** $\leq_p$ **PB**.

Obs0: diferența dintre reducerea polinomială și reducerea Turing este că aici F trebuie să fie determinist polinomial.

Obs1: faptul ca reducerea nu merge (neapărat) și de la PB la PA se vede în sublinierea cuvintelor "orice" și "o". Când reducem PA la PB luăm o intrare oarecare pentru PA și construim în mod convenabil o intrare pentru PB.

Obs2: demonstrația faptului ca o problemă se reduce polinomial la alta nu e completă dacă nu demonstrați **implicația în ambele sensuri**. Întâi trebuie demonstrat că daca I este o instanța-da pentru PA atunci F(I) este o instanța-da pentru PB, apoi ca daca F(I))este o instanța-da pentru PB atunci I este o instanța da pentru PA.

Consecințe ale stabilirii unei relații de reducere polinomială:

$A \leq_p B \wedge B \in P \Rightarrow A \in P$

$A \leq_p B \wedge A \notin P \Rightarrow B \notin P$

Aceste consecințe se folosesc pentru a demonstra apartenența unor probleme la o anumită clasă de probleme, folosind reduceri (de) la probleme a căror (in)tractabilitate este cunoscută.

Clase de probleme

P = { A | $\exists$ B un algoritm determinist polinomial care rezolvă A }

NP = { A | $\exists$ B un algoritm nedeterminist polinomial care rezolvă A }

NP-hard = { A | $\forall$ Q $\in$ NP $\bullet$ Q $\leq_p$ A }

NP-complete = { A | A $\in$ NP $\wedge$ A$\in$ NP-hard }

Strategii de a demonstra apartenența la o clasă de probleme

P: se construiește un algoritm determinist polinomial

NP: se construiește un algoritm nedeterminist polinomial SAU se arată ca soluția poate fi verificată în timp polinomial

NP-hard: se găsește o problema cunoscută ca NP-hard și se reduce această problemă la problema curentă (nu invers!)

NP-complete: se arată că e NP si NP-hard

Alte clase de probleme

PSPACE = clasa problemelor care sunt rezolvate de algoritmi deterministi in timp nelimitat si utilizând spațiu polinomial

NPSPACE = clasa problemelor care sunt rezolvate de algoritmi NEdeterministi în timp nelimitat și utilizând spațiu polinomial

PSPACE = NPSPACE (Savitch, 1970)

Teorema 1. Relația de reducere polinomială $\leq_p$ este tranzitivă.

Teorema 2. O problemă Q2 este NP-hard dacă și numai dacă există o problema NP-completă Q1 astfel încat Q1 $\leq_p$ Q2.

Teorema 3. Problemele NP-complete formează o clasă de echivalență în raport cu relația $\leq_p$ (altfel spus, dacă Q1, Q2 sunt NP-complete atunci Q1 $\leq_p$ Q2 și Q2 $\leq_p$ Q1).

Teorema 4. Dacă există o problemă Q $\in$ NP-completă și Q $\in$ P, atunci P = NP.

Teorema 5. Dacă există o problemă Q$\in$ NP-hard și Q $\in$ P, atunci P = NP.

Demonstrați reducerile polinomiale următoare:

1. 3-Colorare $\leq_p$ 4-Colorare

   Problema **k-Colorare** (k$\geq$2): Dându-se un graf neorientat G = (V, E) și k culori, se pot colora nodurile grafului folosind cele k culori astfel încât niciun nod sa nu aibă un vecin de aceeași culoare?

2. Partitie $\leq_p$ Suma de submulțime / q-Sume

   Problema Partiție: Dându-se un număr n și o mulțime de n întregi A = $\{a_1, ..., a_n\}$, există S $\subset$ {1, ..., n} astfel încât $\sum_{i \in S} a_i = \sum_{i \notin S} a_i$?

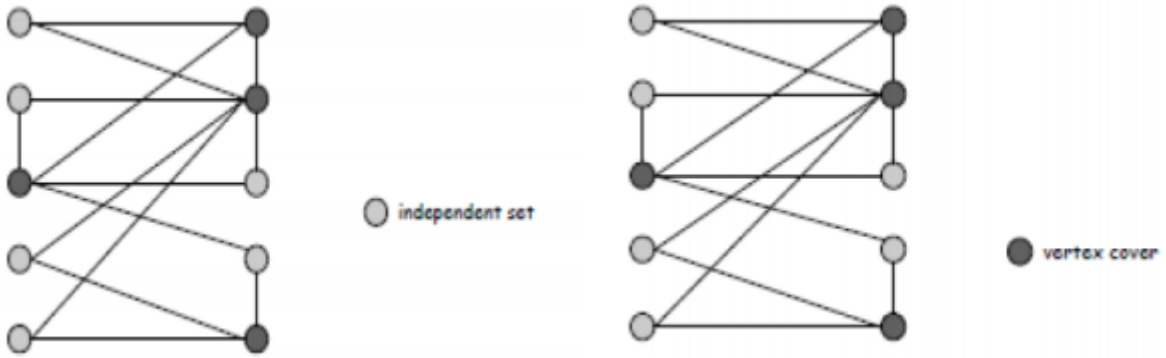
   Problema Suma de submulțime /q-Sume(Subset Sum): Dându-se numerele t, q și o mulțime de t întregi B = $\{b_1, ..., b_t\}$, există $S' \subseteq \{1, ..., t\}$ astfel încât $\sum_{i \in S'} b_i$ = q?

3. Independent Set $\leq_p$ Vertex Cover

   Problema Vertex Cover: Dându-se un graf neorientat G = (V, E) și un număr k, există o submulțime S de noduri, |S| = k, astfel încât fiecare muchie are cel puțin un capăt în S?

   Problema Independent Set: Dându-se un graf neorientat $G' = (V', E')$ și un număr k', există o submulțime S' de noduri, $|S'| = k'$, astfel încât fiecare muchie are cel mult un capăt în S'?

   Exemplu:



4. Vertex Cover $\leq_p$ Set Cover

   Problema Set Cover: Dându-se o mulțime U, o colecție S_1, S_2, ..., S_m de submulțimi ale lui U și un număr k', există o colecție de k' astfel de submulțimi care reunite să dea U?